

Universidad Siglo 21

Ingeniería en Software

Seminario Final

Módulo 1: Introducción y Métodos

Trabajo Final de Graduación: Trabajo de Investigación

Tema de investigación:

**Interfaces cerebro-computadora bajo condiciones adversas: del
software a la decodificación**

Alumno: Agustín Tamagusuku

Legajo: SOF02612

Profesor tutor: Luis Miguel Nuñez

Fecha de entrega: 7 de septiembre de 2026

Introducción

Una interfaz cerebro-computadora (BCI, por su sigla en inglés) es un sistema que mide la actividad del sistema nervioso central y la convierte en una salida artificial, por ejemplo un texto en pantalla o el movimiento de un cursor, que reemplaza o complementa una salida natural del cuerpo (Wolpaw et al., 2002). Tiene aplicaciones en investigación con personas sanas y en neurorrehabilitación, pero uno de sus principales usos clínicos, y el que enmarca este trabajo, es la comunicación asistiva para personas con parálisis severa que conservan la capacidad de pensar pero no la de moverse o hablar. Los resultados recientes muestran hasta dónde llegó el campo: Willett et al. (2023) reportaron una neuroprótesis del habla, un dispositivo implantado que reemplaza la función de hablar, que permitió a una participante con esclerosis lateral amiotrófica, una enfermedad neurodegenerativa que paraliza progresivamente los músculos, producir texto a 62 palabras por minuto a partir de su actividad cortical, Metzger et al. (2023) describieron un sistema que decodificó habla y expresiones faciales para controlar un avatar digital, y Card et al. (2024) mostraron que un sistema comparable pudo calibrarse en pocas sesiones con tasas de error de palabra inferiores al 3 %.

Para entender qué se estudia en este trabajo conviene seguir el recorrido de la señal. Nicolas-Alonso y Gomez-Gil (2012) describieron las etapas de toda BCI: la adquisición, en la que electrodos captan la actividad eléctrica del cerebro, en la variante no invasiva mediante electroencefalografía (EEG) sobre el cuero cabelludo, donde cada electrodo es un canal y cada valor de voltaje que se registra, cientos de veces por segundo, es una muestra; el transporte, que lleva esa señal desde el equipo de registro hasta los programas que la procesan, muchas veces entre procesos o máquinas distintas; el preprocesamiento, que filtra la señal y la segmenta en ventanas, tramos de duración fija

que se analizan de a uno; la decodificación, en la que un modelo entrenado previamente con ejemplos etiquetados reconoce en cada ventana cuál de un conjunto de intenciones está ejecutando la persona; y la decisión, que convierte esa intención en una acción. Un ejemplo sencillo es la imaginería motora (*motor imagery*), que consiste en imaginar un movimiento sin ejecutarlo: la persona imagina mover la mano izquierda o la derecha, ese acto produce patrones distinguibles en el EEG sobre la corteza motora, la región del cerebro que controla el movimiento voluntario, de cada hemisferio, y el decodificador clasifica cada ventana como izquierda o derecha para mover un cursor. Todo ese recorrido ocurre en tiempo real, muchas veces por segundo, y se ejecuta sobre una cadena de software que en la investigación es, en su mayor parte, de código abierto y compartida entre laboratorios. Esa cadena de software, y no el implante ni el modelo, es el objeto de esta investigación; la calidad de las decisiones que produce el decodificador es lo que el usuario percibe como funcionamiento o falla.

Para la persona que usa el sistema, el desempeño del decodificador no es un porcentaje sino una cantidad de comunicación por unidad de tiempo. Wolpaw et al. (2002) reportaron tasas máximas de transferencia de información de 10 a 25 bits por minuto para las BCI de su época y describieron que esa tasa depende de la exactitud de cada selección, del número de opciones disponibles y del tiempo que insume cada decisión; una caída de exactitud se paga, además, con las correcciones que el usuario debe hacer para deshacer las selecciones erróneas. Los sistemas recientes alcanzaron velocidades de comunicación muy superiores en tareas como la decodificación del habla, que se reportan en palabras por minuto y no son directamente comparables con aquellas tasas en bits, pero la relación de fondo se mantiene: una pérdida de exactitud en la decodificación se traduce, en una medida que depende de la tarea y del protocolo, en tiempo y esfuerzo adicionales para

una persona con movilidad muy limitada. Por eso este trabajo mide el efecto de las fallas de software en el desempeño del decodificador y no solo en el estado técnico del sistema.

La decodificación de imaginaria motora tiene métodos y recursos consolidados. Blankertz et al. (2008) sistematizaron los patrones espaciales comunes (CSP, por su sigla en inglés), una técnica que combina los canales del EEG para resaltar las diferencias entre dos clases, habitualmente seguida de un análisis discriminante lineal (LDA), un clasificador que separa las clases con una frontera lineal; Lotte et al. (2018) revisaron diez años de algoritmos de clasificación y concluyeron que esas combinaciones lineales seguían siendo referencias competitivas, y Roy et al. (2019) relevaron el uso creciente de *deep learning* (aprendizaje profundo) advirtiendo sobre la dificultad de comparar resultados entre estudios. Para hacerlos comparables existen recursos públicos: Tangermann et al. (2012) documentaron los datos de la competencia BCI Competition IV, entre ellos el conjunto 2a de imaginaria motora en nueve sujetos; Jayaram y Barachant (2018) desarrollaron MOABB, que estandariza la carga de esos conjuntos y la evaluación de algoritmos, y Gramfort et al. (2013) aportaron MNE-Python como biblioteca de análisis de EEG.

El software que ejecuta el sistema completo también se comparte. Schalk et al. (2004) presentaron BCI2000 y Renard et al. (2010) OpenViBE; más recientemente el ecosistema se desplazó hacia Python con BciPy (Memmott et al., 2021), MEDUSA (Santamaría-Vázquez et al., 2023), Timeflux (Clisson et al., 2019) y el marco con el ser humano en el lazo de Gemborn Nilsson et al. (2023), y Ali et al. (2024) publicaron BRAND para experimentos en lazo cerrado, en los que la salida del sistema realimenta al usuario en tiempo real, sobre datos intracorticales, es decir, registrados con electrodos implantados dentro de la corteza cerebral. Estos ocho artículos describen la arquitectura

y las capacidades de cada herramienta, y varios reportan latencias en condiciones nominales; ninguno reporta el desempeño de un decodificador bajo una condición de falla del software. En la capa de transporte hay un estándar de facto: Lab Streaming Layer (LSL), una biblioteca de código abierto que permite que distintos programas, en una misma máquina o en varias conectadas en red, publiquen y reciban flujos de señal en tiempo real con sus marcas de tiempo sincronizadas, de modo que el equipo que registra el EEG, el que lo procesa y el que presenta los estímulos puedan trabajar como un solo sistema. Kothe et al. (2025) la describieron y reportaron, según su propio relevamiento, más de 2.300 menciones en artículos científicos y la integración en BCI2000, OpenViBE, MNE-Python, Timeflux y MEDUSA. El artículo describe con detalle la arquitectura: los datos viajan por una red local convencional, con UDP, un protocolo sin confirmación de entrega, para descubrir los flujos disponibles, y una conexión TCP, que garantiza la entrega y el orden de los datos, para transmitirlos entre el productor, el programa que emite el flujo, y el consumidor, el que lo recibe; los relojes de las máquinas se sincronizan con un algoritmo derivado del protocolo de tiempo de red (NTP), el mismo principio con el que las computadoras ajustan su hora, con una nueva estimación del desfase cada cinco segundos; los búferes (memorias intermedias que retienen datos mientras el consumidor no puede procesarlos) tienen capacidad configurable, de modo que si un consumidor deja de recibir momentáneamente los datos quedan retenidos hasta que pueda tomarlos; y si un productor desaparece, el consumidor lo busca periódicamente y se reconecta cuando reaparece, incluso en otra máquina con otra dirección. Los autores afirman que la biblioteca se somete periódicamente a pruebas de estrés con cientos de flujos y desconexiones, apagados y reconexiones aleatorios, y reportan como resultado de campo un *jitter* con desviación estándar de aproximadamente 0,5 ms entre flujos de EEG y

electromiografía, el registro de la actividad muscular. Lo que ese artículo no reporta es lo que este trabajo busca medir: qué proporción de muestras se pierde y cuánto tarda la recuperación cuando esos mecanismos actúan, y qué efecto tiene ese comportamiento sobre las etapas posteriores de la cadena, en particular sobre la decodificación. En ese sentido, las propiedades de tolerancia a fallos de LSL son, hasta donde alcanza esta revisión, autorreportadas y cualitativas.

Un relevamiento propio de los repositorios públicos de esas herramientas, realizado el 16 de agosto de 2026 sobre dos indicadores, la cantidad de estrellas de cada repositorio en GitHub y la fecha de su último cambio publicado, completa el cuadro. LSL registraba cambios recientes y una comunidad amplia, en consonancia con su rol de estándar de facto; BciPy y MEDUSA registraban cambios recientes pero comunidades reducidas, de entre veinte y algo más de ciento cincuenta estrellas; Timeflux no registraba cambios desde diciembre de 2024; y BRAND se presentaba como infraestructura de un consorcio académico y no como estándar. Esos indicadores son indirectos y no miden la calidad del software ni la existencia de verificación, pero el panorama que muestran, una capa de transporte concentrada en una herramienta y una capa de aplicación fragmentada entre proyectos pequeños y de actividad desigual, refuerza el interés de contar con un protocolo de evaluación independiente y reutilizable, y sugiere que una debilidad no medida en la capa de transporte alcanzaría a todo el ecosistema a la vez.

Una cadena en tiempo real impone restricciones que el análisis fuera de línea no tiene. Gemborn Nilsson et al. (2023) diseñaron su marco con cuatro módulos, un coordinador central, un módulo de cálculo construido sobre Timeflux que preprocesa, entrena y clasifica, y dos interfaces gráficas, comunicados por LSL para los datos y por *websockets*, un canal de comunicación bidireccional con el navegador, para la

presentación de estímulos; al caracterizar el sistema documentaron que los propios dispositivos periféricos introducen retrasos considerables, entre 20 y 40 ms con 5 ms de *jitter* en un casco de EEG inalámbrico y entre 9 y 117 ms en los monitores, y señalaron como limitación de fondo que una decodificación más avanzada consume un tiempo que obliga a analizar los datos fuera de línea. Wilson et al. (2010) habían definido la latencia de un sistema BCI, el tiempo que tarda un cambio en la entrada en producir un cambio causalmente relacionado en la salida, como la suma de las contribuciones de la conversión analógica-digital, el procesamiento y la salida, más el *jitter* entre bloques; midieron BCI2000 y una implementación en MATLAB sobre cuatro computadoras y dos sistemas operativos, encontraron latencias de procesamiento de hasta 50 ms, salidas de video de 5 a 20 ms según el monitor y el sistema operativo, y un rango de 8 a 141 ms en MATLAB, y concluyeron que la elección del hardware y del sistema operativo tiene un efecto significativo sobre la temporalidad del experimento. Ambos trabajos miden sistemas que funcionan correctamente; ninguno introduce una falla para observar cómo la cadena responde. De ahí se toman las tres propiedades del flujo de datos que este trabajo mide: la integridad, que todas las muestras emitidas lleguen; la temporalidad, que lleguen cuando corresponde y a intervalos regulares; y la disponibilidad, que el flujo no se interrumpa. La pérdida de muestras (*sample loss*) compromete la primera, el retraso y el *jitter* (llegada irregular de los datos) la segunda, y la desconexión la tercera. La literatura sobre robustez de la decodificación, en cambio, perturbó la señal y no el software: Urigüen y Garcia-Zapirain (2015) revisaron la eliminación de artefactos del EEG, es decir, de contaminaciones no cerebrales de la señal como parpadeos o movimientos musculares, y Meng et al. (2018) estudiaron cómo la reducción del número de electrodos

afectó el desempeño de una BCI en línea; en ambos casos el transporte se tomó como dado.

Esas tres propiedades no se comprometen solo en condiciones excepcionales. LSL transporta los datos por red entre procesos de un mismo equipo o entre máquinas distintas (Kothe et al., 2025), y los equipos de registro se conectan al ordenador por enlaces cableados o inalámbricos. En una arquitectura así, una red congestionada, un dispositivo inalámbrico que pierde alcance por unos instantes, un proceso consumidor que se demora porque la máquina está ocupada o una conexión que se corta y se restablece son condiciones ordinarias de operación. Ocurren, sin embargo, en capas distintas del sistema. La pérdida de alcance de un dispositivo inalámbrico afecta a la adquisición, antes de que la señal entre en LSL, y puede eliminar muestras. La congestión de la red afecta al transporte, donde el protocolo TCP que LSL utiliza retransmite lo que se pierde, de modo que el efecto no es una muestra faltante sino más latencia y más variabilidad. Un consumidor demorado afecta al procesamiento: las muestras se acumulan en un búfer de capacidad finita que, según la documentación de LSL, descarta las más antiguas cuando se llena (Lab Streaming Layer, s. f.). Todas esas situaciones se manifiestan en el mismo lugar, la interfaz por la que el pipeline recibe el flujo, como muestras que faltan, que llegan tarde o a intervalos irregulares, o que dejan de llegar por un tiempo. Los cuatro modelos de fallo de esta investigación, pérdida de muestras, *jitter*, retraso y desconexión, se definen sobre esa interfaz: son perturbaciones observables del flujo que entra al pipeline, con orígenes posibles en más de una capa, y no se presentan como fallas propias del protocolo LSL.

Fuera del dominio BCI, la ingeniería de software tiene un cuerpo de conocimiento maduro para este tipo de pregunta. Avizienis et al. (2004) fijaron la taxonomía de la

computación confiable (*dependable computing*), con la distinción entre falla, error y fallo de servicio, describieron la tolerancia a fallos como detección de errores más recuperación, e incluyeron los fallos en los que el sistema no señala la degradación de su servicio (*silent failures*). Natella et al. (2016) sistematizaron la inyección de fallos de software como el método que anticipa los escenarios de peor caso provocados por software defectuoso mediante la introducción deliberada de fallas, revisaron cómo evolucionaron sus técnicas en torno a tres criterios, la representatividad de las fallas inyectadas respecto de las reales, la eficiencia de las campañas y la usabilidad de las herramientas, y documentaron sus aplicaciones a la evaluación de sistemas tolerantes a fallos. Basiri et al. (2016) describieron cómo Netflix llevó esa idea a sistemas distribuidos en producción bajo el nombre de *chaos engineering* (ingeniería del caos), a partir de un servicio interno que apagaba al azar instancias de sus propios servidores para obligar a diseñar software que tolerara esas pérdidas, y la formalizaron en cuatro principios: formular una hipótesis sobre el comportamiento estable del sistema, variar eventos del mundo real, correr los experimentos en el entorno real y automatizarlos para que corran de forma continua. Los dos primeros principios son los que esta investigación adopta: una hipótesis sobre el estado estable, que aquí es el desempeño del pipeline sin fallos, y perturbaciones que reproducen eventos ordinarios de operación.

Los sistemas de procesamiento de flujos (*stream processing*), que como una BCI operan sobre secuencias continuas de eventos con restricciones temporales, fueron objeto de evaluaciones de este tipo. Karimov et al. (2018) compararon plataformas distribuidas con cargas de trabajo controladas en rendimiento y latencia, y Vogel et al. (2024) llevaron la comparación al terreno de la recuperación ante fallos: sobre Flink, Kafka Streams y Spark Structured Streaming desplegados en Kubernetes, inyectaron con la herramienta

Chaos Mesh la eliminación y la caída de instancias a intervalos regulares, y midieron el rendimiento, la latencia de los eventos en percentiles, el retraso acumulado del consumidor y el tiempo de recuperación, detectado automáticamente a partir de la media móvil y la desviación de las series. Encontraron que Flink fue el más estable y el de recuperación más rápida, que Kafka Streams mostró inestabilidad después de las fallas por su estrategia de rebalanceo, la forma en que redistribuye el trabajo entre las instancias que quedan, con casos en los que no se recuperó por completo antes de la falla siguiente, y que Spark se recuperó de manera consistente pero con mayor latencia. Ese trabajo ofrece el molde metodológico que esta investigación adapta: familias de fallo, niveles de severidad, condición de referencia sin fallo, métricas de sistema medidas en el tiempo y un criterio explícito de recuperación. Lo que no tiene, porque su objeto no lo requiere, es un modelo de decodificación al final de la cadena cuya salida pueda contrastarse con una verdad conocida.

De esos tres grupos de fuentes, plataformas de BCI, robustez de la decodificación e inyección de fallos en sistemas de propósito general, ninguno combina una falla inyectada en el transporte de señal con una medición simultánea del estado del sistema y del desempeño de un decodificador contra etiquetas conocidas. Es posible que esa combinación exista en fuentes no relevadas; lo que puede afirmarse es que no se encontró en las fuentes de referencia relevadas para este trabajo. Estudiarla no requiere hardware: MOABB provee la carga de los conjuntos públicos y el ecosistema de MNE-Python incluye componentes que reproducen una grabación como flujo LSL en tiempo real (Gramfort et al., 2013; Kothe et al., 2025), de modo que el mismo software que un laboratorio usaría con un amplificador y una persona puede ejecutarse con una fuente reproducible y variar solo aquello que se quiere estudiar.

El problema que aborda esta investigación es el desconocimiento sobre qué le ocurre a la decodificación de una BCI cuando el software que transporta la señal falla. No se sabe con qué magnitud la pérdida de muestras, el *jitter*, el retraso y la desconexión alteran la integridad, la temporalidad y la disponibilidad del flujo que recibe el decodificador, ni a partir de qué severidad esas alteraciones se traducen en una caída del desempeño de clasificación. Tampoco se sabe si hay condiciones en las que el proceso sigue activo, el flujo conectado y sin excepciones mientras la calidad de las decisiones cae, ni si la telemetría del propio sistema, las mediciones que registra sobre su propio estado, alcanza para detectar o anticipar esa caída. Este último punto es el que más pesa cuando el usuario depende del sistema para comunicarse: una falla que el sistema informa puede gestionarse desde afuera; una que no informa solo se descubre por sus consecuencias.

La pregunta central es cómo se propagan fallos controlados de la capa de transporte de señal hacia las propiedades operativas y el desempeño funcional de un pipeline BCI en tiempo real, y en qué medida la telemetría del sistema permite detectar o anticipar esa degradación. De ella se desprenden tres interrogantes, que se corresponden con las tres comparaciones del diseño experimental: ¿cómo afectan distintos tipos y severidades de fallo a la integridad de los datos, la disponibilidad, la latencia y el desempeño de decodificación del pipeline?; ¿existen condiciones de fallo bajo las cuales el estado observable de la infraestructura y el desempeño del decodificador divergen?; y ¿puede un modelo entrenado con telemetría multivariable detectar o anticipar la degradación funcional mejor que un detector basado en umbrales? Las tres admiten un resultado negativo: si la infraestructura absorbe los fallos, se documenta esa tolerancia; si no hay divergencia, ese resultado vale; y si un detector por umbrales rinde igual o mejor

que el modelo, la complejidad adicional no se justifica. Asumir lo contrario de antemano habría condicionado el experimento hacia el resultado esperado.

La relevancia del problema proviene del lugar del software abierto en el campo: cuando una herramienta es gratuita, ubicua y la base sobre la que se validan y replican los resultados, una propiedad no medida de esa herramienta es una incertidumbre que arrastran todos los trabajos que corren sobre ella, y en una BCI del otro lado hay una persona que con frecuencia no dispone de otro canal para corregir una decisión errónea. Las BCI son una de las fronteras de la transformación digital en salud, y que su infraestructura sea confiable es condición para que lo construido sobre ella sea reproducible. El planteo es conservador: somete al software a las condiciones que sus autores afirman tolerar, con datos públicos y sin hardware de adquisición, unidades de procesamiento gráfico ni acceso a pacientes. Toma de Vogel et al. (2024) la estructura del protocolo, de Natella et al. (2016) los criterios de diseño y de Basiri et al. (2016) la idea de verificar lo declarado; se aparta de ellos en el objeto, un pipeline BCI, y en la variable de respuesta, que incluye el desempeño de un decodificador. Sigue la tradición de la experimentación controlada en ingeniería de software (Wohlin et al., 2012), en la que el artefacto construido sirve para medir y no es el resultado. Lo que se espera obtener es un protocolo reproducible con su banco de pruebas abierto, las curvas que relacionen cada condición de falla con su efecto sobre la infraestructura y la decodificación, y una comparación entre un mecanismo de alerta por umbrales y un modelo basado en telemetría. Conviene dejar claro desde ahora qué no pretende el estudio. El enfoque Software-in-the-Loop reproduce el comportamiento del software real con señal grabada, pero no reproduce los electrodos, los amplificadores, la interferencia electromagnética ni la adaptación de la persona a un sistema que le devuelve retroalimentación. Las

conclusiones se refieren, por lo tanto, al comportamiento del pipeline de software bajo las condiciones estudiadas y no a la resiliencia de una BCI clínica completa; extender el protocolo a hardware en el lazo queda como trabajo futuro.

El objetivo general de esta investigación es caracterizar empíricamente cómo se propagan fallos controlados de la capa de transporte de señal, en particular la pérdida de muestras, el *jitter*, el retraso y la desconexión, hacia las propiedades operativas y el desempeño de decodificación de un pipeline BCI en tiempo real basado en software de código abierto, y evaluar en qué medida la telemetría del sistema permite detectar o anticipar esa degradación.

De ese objetivo general se derivan los siguientes objetivos específicos. Construir un banco de experimentación Software-in-the-Loop, es decir, el sistema de software real alimentado con señal grabada en lugar de una persona, que reproduzca señal electroencefalográfica pública como flujo en tiempo real sobre Lab Streaming Layer, con inyectores de fallos parametrizables, un decodificador de referencia y un recolector de telemetría. Cuantificar el efecto de cada tipo y severidad de fallo sobre la integridad, la temporalidad y la disponibilidad del flujo de datos, en comparación con la condición de referencia sin fallos. Medir el efecto de esos mismos fallos sobre el desempeño del decodificador de referencia y describir las curvas de degradación funcional resultantes. Identificar las condiciones de fallo bajo las cuales el estado observable de la infraestructura y el desempeño funcional del decodificador divergen. Comparar la capacidad de un detector basado en umbrales y la de un modelo de clasificación entrenado con telemetría multivariable para detectar o anticipar la degradación funcional. Documentar el protocolo, los inyectores y los datos generados de manera que el estudio pueda replicarse y extenderse a otros pipelines.

Métodos

Diseño

La investigación adopta un enfoque cuantitativo, con alcance explicativo sobre una base descriptiva: manipula deliberadamente las condiciones de falla para observar su efecto sobre variables de respuesta medidas numéricamente (Hernández Sampieri et al., 2014). El diseño es experimental, de medidas repetidas, con condición de control. El objeto de estudio es un pipeline BCI único; las unidades experimentales son los nueve sujetos del conjunto de datos, cada uno con dos sesiones: la primera entrena el decodificador sin fallos y fuera de línea, y la segunda se reproduce a través del pipeline bajo la condición de referencia y bajo cada condición de falla. Las observaciones son los ensayos clasificados en cada reproducción, y las comparaciones se hacen dentro de cada sujeto y entre condiciones. Las variables independientes son el tipo de fallo, con cuatro niveles (pérdida de muestras, *jitter*, retraso y desconexión), y su severidad, con tres niveles por tipo. Se mantienen constantes la señal, el modelo, su configuración, el preprocesamiento y la máquina, y el orden de las condiciones se aleatoriza.

Cada ejecución reproduce una corrida completa de la sesión de evaluación, de 48 ensayos y del orden de siete minutos en tiempo real. De las seis corridas de cada sujeto, las cinco primeras integran la campaña principal y la sexta se reserva para la prueba piloto, de modo que los datos con los que se calibran severidades, umbral y tamaño de ventana no participan de las comparaciones. El diseño es por bloques: cada una de las cinco corridas se reproduce sin fallo y bajo cada condición de falla, con una realización del fallo fijada por una semilla aleatoria, el valor que determina la secuencia de números aleatorios y hace repetible la ejecución. Toda comparación se hace, así, entre la misma corrida con y sin fallo, lo que separa el efecto de la falla de las diferencias naturales de

clasificación entre corridas; las cinco corridas no son repeticiones idénticas sino bloques que permiten estimar si el efecto se sostiene con datos distintos. La campaña principal comprende 585 ejecuciones, cinco de referencia y sesenta con fallo por cada uno de los nueve sujetos, unas 68 horas de reproducción en tiempo real que el ejecutor de experimentos automatiza; el piloto se ejecuta sobre la sexta corrida de un subconjunto de sujetos.

Participantes

La investigación no involucra participantes humanos, por lo que no corresponde consentimiento informado. La población de referencia son los pipelines BCI construidos sobre el ecosistema de código abierto cuyo transporte es LSL; la muestra, no probabilística e intencional, es un pipeline de referencia compuesto por MNE-LSL para la reproducción de la señal, código propio para inyección de fallos y telemetría, y un decodificador CSP más LDA sobre MNE-Python y scikit-learn. Se prefirió esa composición a un marco de aplicación completo como BciPy o MEDUSA porque los marcos incorporan sus propias capas de búfer y manejo de errores, y con ellos no sería posible atribuir un efecto al transporte o al marco; una segunda implementación sobre uno de esos marcos se contempla como validez externa, después de la campaña principal. Las conclusiones se formulan para el pipeline y las condiciones estudiadas.

La señal proviene del conjunto público BCI Competition IV 2a, BNCI2014_001 en MOABB: EEG de 22 canales, es decir, 22 electrodos, y 3 canales de electrooculografía, que registran el movimiento ocular para identificar artefactos, a 250 Hz, o sea 250 muestras por segundo y por canal, nueve sujetos, dos sesiones de seis corridas de 48 ensayos, doce por clase, donde cada ensayo es una repetición de la tarea y cada corrida un bloque de ensayos, en una tarea de imaginería motora de cuatro clases

con ensayos de cuatro segundos (Tangermann et al., 2012). La decodificación se restringe a mano izquierda y mano derecha porque CSP fue formulado para problemas binarios (Blankertz et al., 2008) y sus extensiones multiclase agregan una variabilidad del clasificador ajena a la pregunta. Cada corrida se reproduce completa, con sus 48 ensayos y pausas, para conservar la carga y la estructura temporal originales; el pipeline clasifica solo los 24 ensayos de mano izquierda o derecha, que son los que ingresan al cálculo de desempeño. Se excluyen los ensayos marcados por los autores del conjunto como contaminados por artefactos.

Instrumentos

El instrumento principal es el banco de experimentación Software-in-the-Loop, publicado con licencia abierta, con seis componentes: un servicio de reproducción que emite la señal como flujo LSL a su tasa original; un inyector de fallos entre fuente y consumidor; el pipeline bajo prueba, que segmenta ventanas, preprocesa, extrae características con CSP y clasifica con LDA, combinación elegida por ser el método de referencia sobre este conjunto (Tangermann et al., 2012), seguir siendo competitiva (Lotte et al., 2018), ser determinista y tener costo compatible con el tiempo real; un recolector de telemetría que registra por ventana el estado del flujo y del proceso; dos detectores de salud, uno por umbrales y otro basado en un modelo de clasificación; y un ejecutor que orquesta las campañas, fija semillas y persiste resultados junto con las versiones de software y hardware.

Los modelos de fallo se definen, siguiendo a Avizienis et al. (2004), como fallas que se manifiestan en la interfaz de transporte del pipeline y que pueden o no propagarse como errores en su estado y como fallos de servicio en la salida del decodificador. La

Tabla 1 resume, para cada modelo, dónde se aplica, con qué severidades, qué propiedad mide y qué situación real representa.

Tabla 1

Modelos de fallo: aplicación, severidades exploratorias, propiedad medida y situación real que representan

Modelo	Dónde se aplica	Severidades	Qué mide	Situación real
Pérdida de muestras	Contenido del flujo: el inyector omite muestras, al azar o en ráfagas	1 %, 5 %, 10 % de las muestras	Tolerancia del pipeline a datos faltantes	Dispositivo que pierde muestras antes de LSL; búfer del consumidor que descarta las más antiguas al saturarse
<i>Jitter</i>	Instante de entrega de cada bloque: perturbación aleatoria	Desviación de 10, 50, 100 ms	Tolerancia a la irregularidad temporal	Congestión de red; retransmisiones de TCP; consumidor demorado
Retraso	Instante de entrega de cada bloque: desplazamiento constante	50, 100, 250 ms	Tolerancia a la latencia	Los mismos orígenes que el <i>jitter</i> , con efecto sostenido
Desconexión	Transporte real: el flujo de origen se cierra y se vuelve a crear	0,5, 1, 3 s hasta la reconexión	Reconexión y búferes que LSL declara (único modelo que los ejercita directamente)	Corte del enlace; caída del proceso productor

Nota. Los rangos toman como referencia el ensayo de cuatro segundos a 250 Hz (mil muestras): una desconexión de 0,5 s elimina una octava parte de un ensayo y una de 3 s casi lo elimina; los retrasos van del orden de un bloque de transmisión a un cuarto de segundo; el *jitter* se ubica por debajo, en el orden y por encima de la duración de un bloque; la pérdida va de una fracción pequeña al equivalente a cien muestras por ensayo.

Conviene subrayar la distinción que la tabla recoge: LSL transmite sobre TCP, de modo que una pérdida de paquetes de red no elimina muestras sino que las demora; por eso la pérdida de muestras representa lo que ocurre antes de LSL o en un búfer saturado, mientras que el *jitter* y el retraso son el síntoma propio de la red y del consumidor, y solo

la desconexión pone a prueba los mecanismos de recuperación del propio LSL. Los valores definitivos de severidad se fijan tras la prueba piloto y no se modifican después de observar resultados.

Las variables dependientes se agrupan en cinco familias. Integridad: relación entre muestras recibidas y esperadas, número y duración de huecos, pérdida total y en ráfagas. Temporalidad: latencia de extremo a extremo, media y desviación del intervalo entre llegadas, tiempo de recuperación. Disponibilidad: continuidad del flujo, reconexiones, proporción de ventanas descartadas. Desempeño funcional: *balanced accuracy* (exactitud balanceada, el promedio de la exactitud obtenida en cada clase) por sujeto y condición como métrica principal, porque las condiciones de falla descartan ventanas de manera no uniforme y desbalancean el conjunto evaluado, complementada con exactitud simple, confianza media y tasa de decisiones válidas. Observabilidad: para cada detector, tasa de verdaderos positivos, tasa de falsos positivos y retardo de detección (*detection delay*). Se define como estado operativo la conjunción de proceso activo, flujo conectado, ausencia de excepciones y llegada continua de muestras y predicciones; y como degradación funcional, una caída de la *balanced accuracy* que supere el ruido de la referencia del mismo sujeto. Para que ambas magnitudes estén en la misma escala, la *balanced accuracy* se calcula siempre sobre una ventana móvil (*rolling window*) de ensayos consecutivos: la distribución de referencia se obtiene aplicando esa misma ventana a las cinco corridas sin fallo del sujeto, y el umbral se fija a partir de su dispersión. El tamaño de la ventana se fija durante la prueba piloto y permanece constante en la campaña principal. El inicio de la degradación es la primera ventana que cruza el umbral, y respecto de él se mide el retardo de detección.

Análisis de datos

El análisis es cuantitativo y se realiza en Python con las bibliotecas pandas, SciPy y scikit-learn, de manejo de datos, estadística y aprendizaje automático respectivamente. Se describen las variables por sujeto, tipo y severidad y se construyen curvas de degradación por variable. La unidad estadística es el sujeto. Para evitar pseudorreplicación, las cinco corridas de cada sujeto y condición se resumen en una sola observación por sujeto, la mediana de la métrica sobre las cinco corridas, y son esas nueve observaciones por condición las que ingresan a las pruebas; la variabilidad entre corridas se reporta de manera descriptiva. Para el primer interrogante, cada condición de falla se compara con la de referencia mediante pruebas estadísticas no paramétricas para muestras apareadas, que no asumen una distribución normal y son adecuadas para el número reducido de sujetos: una prueba de Friedman sobre la referencia y las tres severidades de cada tipo de fallo, seguida de comparaciones de Wilcoxon de cada severidad contra la referencia con corrección de Holm por comparaciones múltiples, y reporte de tamaños de efecto. Para el segundo, se cruzan ventana a ventana el estado operativo y la degradación funcional y se cuantifica la proporción de ventanas en que divergen, por condición. Para el tercero, el detector por umbrales y el modelo se evalúan sobre las mismas ventanas etiquetadas por su estado funcional medido, con validación cruzada que deja fuera un sujeto completo en cada iteración, es decir, entrenando con ocho sujetos y probando con el restante, para evitar *data leakage* (filtración de información entre los datos de entrenamiento y los de prueba), y se comparan por precisión, sensibilidad, puntaje F1 y retardo de detección. Los modelos candidatos son la regresión logística, como referencia lineal e interpretable, y los algoritmos *random forest* (bosques aleatorios) y *gradient boosting* (aumento de gradiente), porque la hipótesis del tercer interrogante es que la degradación se anticipa por combinaciones de variables que un umbral individual no

captura; las arquitecturas profundas se excluyen por el volumen de datos disponible.

Datos brutos, guiones y resultados intermedios se publican con el banco de pruebas.

Referencias

- Ali, Y. H., Bodkin, K., Rigotti-Thompson, M., Patel, K., Card, N. S., Bhaduri, B., Nason-Tomaszewski, S. R., Mifsud, D. M., Hou, X., Nicolas, C., Allcroft, S., Hochberg, L. R., Au Yong, N., Stavisky, S. D., Miller, L. E., Brandman, D. M., y Pandarinath, C. (2024). BRAND: A platform for closed-loop experiments with deep network models. *Journal of Neural Engineering*, 21(2), 026046.
<https://doi.org/10.1088/1741-2552/ad3b3a>
- Avizienis, A., Laprie, J.-C., Randell, B., y Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1), 11-33.
<https://doi.org/10.1109/TDSC.2004.2>
- Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., y Rosenthal, C. (2016). Chaos engineering. *IEEE Software*, 33(3), 35-41.
<https://doi.org/10.1109/MS.2016.60>
- Blankertz, B., Tomioka, R., Lemm, S., Kawanabe, M., y Müller, K.-R. (2008). Optimizing spatial filters for robust EEG single-trial analysis. *IEEE Signal Processing Magazine*, 25(1), 41-56. <https://doi.org/10.1109/MSP.2008.4408441>
- Card, N. S., Wairagkar, M., Iacobacci, C., Hou, X., Singer-Clark, T., Willett, F. R., Kunz, E. M., Fan, C., Vahdati Nia, M., Deo, D. R., Srinivasan, A., Choi, E. Y., Glasser, M. F., Hochberg, L. R., Henderson, J. M., Shahlaie, K., Stavisky, S. D., y Brandman, D. M. (2024). An accurate and rapidly calibrating speech neuroprosthesis. *New England Journal of Medicine*, 391(7), 609-618.
<https://doi.org/10.1056/NEJMoa2314132>

- Clisson, P., Bertrand-Lalo, R., Congedo, M., Victor-Thomas, G., y Chatel-Goldman, J. (2019). Timeflux: An open-source framework for the acquisition and near real-time processing of signal streams. En *Proceedings of the 8th Graz Brain-Computer Interface Conference 2019*. Verlag der Technischen Universität Graz.
- Gemborn Nilsson, M., Tufvesson, P., Heskebeck, F., y Johansson, M. (2023). An open-source human-in-the-loop BCI research framework: Method and design. *Frontiers in Human Neuroscience*, 17, 1129362.
<https://doi.org/10.3389/fnhum.2023.1129362>
- Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj, R., Jas, M., Brooks, T., Parkkonen, L., y Hämäläinen, M. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, 7, 267.
<https://doi.org/10.3389/fnins.2013.00267>
- Hernández Sampieri, R., Fernández Collado, C., y Baptista Lucio, M. del P. (2014). *Metodología de la investigación* (6.^a ed.). McGraw-Hill.
- Jayaram, V., y Barachant, A. (2018). MOABB: Trustworthy algorithm benchmarking for BCIs. *Journal of Neural Engineering*, 15(6), 066011.
<https://doi.org/10.1088/1741-2552/aadea0>
- Karimov, J., Rabl, T., Katsifodimos, A., Samarev, R., Heiskanen, H., y Markl, V. (2018). Benchmarking distributed stream data processing systems. En *2018 IEEE 34th International Conference on Data Engineering (ICDE)* (pp. 1507-1518). IEEE. <https://doi.org/10.1109/ICDE.2018.00169>
- Kothe, C., Shirazi, S. Y., Stenner, T., Medine, D., Boulay, C., Grivich, M. I., Artoni, F., Mullen, T., Delorme, A., y Makeig, S. (2025). The lab streaming layer for

synchronized multimodal recording. *Imaging Neuroscience*, 3.

<https://doi.org/10.1162/IMAG.a.136>

Lab Streaming Layer. (s. f.). *Frequently asked questions*. Lab Streaming Layer documentation. <https://labstreaminglayer.readthedocs.io/info/faqs.html>

Lotte, F., Bougrain, L., Cichocki, A., Clerc, M., Congedo, M., Rakotomamonjy, A., y Yger, F. (2018). A review of classification algorithms for EEG-based brain-computer interfaces: A 10 year update. *Journal of Neural Engineering*, 15(3), 031005. <https://doi.org/10.1088/1741-2552/aab2f2>

Memmott, T., Koçanaoğulları, A., Lawhead, M., Klee, D., Dudy, S., Fried-Oken, M., y Oken, B. (2021). BciPy: Brain-computer interface software in Python. *Brain-Computer Interfaces*, 8(4), 137-153. <https://doi.org/10.1080/2326263X.2021.1878727>

Meng, J., Edelman, B. J., Olsoe, J., Jacobs, G., Zhang, S., Beyko, A., y He, B. (2018). A study of the effects of electrode number and decoding algorithm on online EEG-based BCI behavioral performance. *Frontiers in Neuroscience*, 12, 227. <https://doi.org/10.3389/fnins.2018.00227>

Metzger, S. L., Littlejohn, K. T., Silva, A. B., Moses, D. A., Seaton, M. P., Wang, R., Dougherty, M. E., Liu, J. R., Wu, P., Berger, M. A., Zhuravleva, I., Tu-Chan, A., Ganguly, K., Anumanchipalli, G. K., y Chang, E. F. (2023). A high-performance neuroprosthesis for speech decoding and avatar control. *Nature*, 620(7976), 1037-1046. <https://doi.org/10.1038/s41586-023-06443-4>

Natella, R., Cotroneo, D., y Madeira, H. S. (2016). Assessing dependability with software fault injection: A survey. *ACM Computing Surveys*, 48(3), 1-55. <https://doi.org/10.1145/2841425>

Nicolas-Alonso, L. F., y Gomez-Gil, J. (2012). Brain computer interfaces, a review.

Sensors, 12(2), 1211-1279. <https://doi.org/10.3390/s120201211>

Renard, Y., Lotte, F., Gibert, G., Congedo, M., Maby, E., Delannoy, V., Bertrand, O., y

Lécuyer, A. (2010). OpenViBE: An open-source software platform to design, test, and use brain-computer interfaces in real and virtual environments.

Presence: Teleoperators and Virtual Environments, 19(1), 35-53.

<https://doi.org/10.1162/pres.19.1.35>

Roy, Y., Banville, H., Albuquerque, I., Gramfort, A., Falk, T. H., y Faubert, J. (2019).

Deep learning-based electroencephalography analysis: A systematic review.

Journal of Neural Engineering, 16(5), 051001. <https://doi.org/10.1088/1741-2552/ab260c>

Santamaría-Vázquez, E., Martínez-Cagigal, V., Marcos-Martínez, D., Rodríguez-

González, V., Pérez-Velasco, S., Moreno-Calderón, S., y Hornero, R. (2023).

MEDUSA©: A novel Python-based software ecosystem to accelerate brain-computer interface and cognitive neuroscience research. *Computer Methods and Programs in Biomedicine*, 230, 107357.

<https://doi.org/10.1016/j.cmpb.2023.107357>

Schalk, G., McFarland, D. J., Hinterberger, T., Birbaumer, N., y Wolpaw, J. R. (2004).

BCI2000: A general-purpose brain-computer interface (BCI) system. *IEEE*

Transactions on Biomedical Engineering, 51(6), 1034-1043.

<https://doi.org/10.1109/TBME.2004.827072>

Tangemann, M., Müller, K.-R., Aertsen, A., Birbaumer, N., Braun, C., Brunner, C.,

Leeb, R., Mehring, C., Miller, K. J., Müller-Putz, G. R., Nolte, G., Pfurtscheller,

G., Preissl, H., Schalk, G., Schlögl, A., Vidaurre, C., Waldert, S., y Blankertz, B.

- (2012). Review of the BCI Competition IV. *Frontiers in Neuroscience*, 6, 55.
<https://doi.org/10.3389/fnins.2012.00055>
- Urigüen, J. A., y Garcia-Zapirain, B. (2015). EEG artifact removal—state-of-the-art and guidelines. *Journal of Neural Engineering*, 12(3), 031001.
<https://doi.org/10.1088/1741-2560/12/3/031001>
- Vogel, A., Henning, S., Perez-Wohlfeil, E., Ertl, O., y Rabiser, R. (2024). A comprehensive benchmarking analysis of fault recovery in stream processing frameworks. En *Proceedings of the 18th ACM International Conference on Distributed and Event-Based Systems (DEBS '24)* (pp. 171-182). ACM.
<https://doi.org/10.1145/3629104.3666040>
- Willett, F. R., Kunz, E. M., Fan, C., Avansino, D. T., Wilson, G. H., Choi, E. Y., Kamdar, F., Glasser, M. F., Hochberg, L. R., Druckmann, S., Shenoy, K. V., y Henderson, J. M. (2023). A high-performance speech neuroprosthesis. *Nature*, 620(7976), 1031-1036. <https://doi.org/10.1038/s41586-023-06377-x>
- Wilson, J. A., Mellinger, J., Schalk, G., y Williams, J. (2010). A procedure for measuring latencies in brain-computer interfaces. *IEEE Transactions on Biomedical Engineering*, 57(7), 1785-1797.
<https://doi.org/10.1109/TBME.2010.2047259>
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., y Wesslén, A. (2012). *Experimentation in software engineering*. Springer. <https://doi.org/10.1007/978-3-642-29044-2>
- Wolpaw, J. R., Birbaumer, N., McFarland, D. J., Pfurtscheller, G., y Vaughan, T. M. (2002). Brain-computer interfaces for communication and control. *Clinical*

Neurophysiology, 113(6), 767-791. [https://doi.org/10.1016/S1388-2457\(02\)00057-3](https://doi.org/10.1016/S1388-2457(02)00057-3)